西安电子科技大学 2025 年数学建模校内赛

承 诺 与 产 权 转 让 书

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上咨询等）与队外的任何人研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果是违反竞赛规则的，如果引用别人的成果或其他公开的资料（包括网上查到的资料），必须按照规定的参考文献的表述方式在正文引用处和参考文献中明确列出。

我们郑重承诺，严格遵守竞赛规则，以保证竞赛的公正、公平性。如有违反竞赛规则的行为，我们将受到严肃处理。

我们同意将参赛论文以及支撑材料中的所建模型、算法以及程序产权归属西安电子科技大学所有。数学与统计实训中心可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

我们参赛选择的题号是（从 A/B/C 中选择一项填写）：C

参赛报名队号为25442

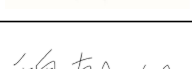
队长所属学院（书院）（请填写完整的全名）：网络与信息安全学院

参赛队员姓名与学号 (此处附电子签名) ：

1. 韩苏玉 24189100106



2. 刘华艺 23009290076



3. 魏朝旭 23009201082

日期：2025 年 5 月 3 日

西安电子科技大学 2025 年大学生数学建模校内赛

评 阅 专 用 页

	评阅人 1	评阅人 2	评阅人 3	总评
成绩				

基于光谱技术的农药残留检测模型

摘要

随着食品安全问题的日益严重，农药残留检测成为了确保公众健康的重要手段。基于光谱技术的检测方法具有无损、高效、灵敏等优点，但在实际应用中，光谱数据常常受到噪声、测量物差异以及批次间分布不一致等问题的影响。本文围绕这些问题，根据题目给定的背景与数据信息，通过降噪处理、特征提取构建分类模型并进行优化，旨在提高检测的准确性和可靠性。

对于问题一, 需要解决数据噪声干扰对光谱数据准确性的影响。光谱数据中紫外和红外波段常存在高频噪声，这会掩盖农药残留的关键特征。为此，本文采用了 **Savitzky-Golay 平滑滤波器**进行降噪处理，并引入**信噪比 (SNR)** 对模型参数进行对比。实验结果表明，参数组合窗口宽度 w=9、阶数 p=4 降噪效果最佳。经过降噪处理后，噪声干扰显著降低，样本光谱也显著平滑，所得相对残差进一步说明模型能在有效去除噪声的基础上，保留重要的光谱特征。

对于问题二, 由于不同测量物的反射率在光谱特征上有所差异，本题研究的关键是如何从高维光谱数据中提取有效特征并进行测量物分类。对此，本题基于**光谱波段统计特征**选取各波段均值、标准差、极差共九个特征，同时选择**支持向量机 (SVM)** 作为分类模型，在验证集达到 100% 的准确率，证明了所提出的特征提取方法具有较好的泛化能力和分类性能。

对于问题三, 需要识别不同浓度农药残留的光谱特征，并基于这些特征对农药喷洒次数进行分类。针对光谱数据高维小样本的特性，本题沿用**支持向量机（SVM）**建立分类模型，并进一步优化 SVM 模型参数。对比实验结果表明，SVM 模型在测试集上的分类准确率高达 95.8%，在各项指标上均表现最优，表明该方法能够准确区分农药喷洒次数。

对于问题四, 考虑了批次间数据分布的不一致性。由于不同批次的样本可能受到环境因素、测量设备等多方面的影响，导致数据的分布存在显著差异。为解决这一问题，本题采用**核主成分回归（PLS-DA）和 XGBoost 混合模型**，并结合二阶差分、局部方差和小波能量等鲁棒性特征提取策略，构建了一个跨批次的通用分类模型。经过优化后，模型在不同批次数据上的准确率均稳定在 95% 以上，表明该方法具有良好的跨批次适应性和高效的分类性能。

关键字：**农药残留　光谱检测　Savitzky-Golay 滤波器　支持向量机　XGBoost**

一、 问题重述

1.1 问题背景

食品安全事关人民群众的身体健康和生命安全，已然成为了衡量社会文明水平的重要标准，是社会稳定的重要基石。如三鹿奶粉中含有三聚氰胺、罐车卸完煤制油直接装运食用油等，食品安全事故的频发严重影响了公众健康和社会秩序。其中农药残留超标的问题极大地增加了食品污染的风险，是威胁消费者的健康的隐形杀手。根据农业农村部数据显示，2024 年第一次国家农产品质量安全例行监测总体合格率为 98%，但是氧乐果、克百威等高风险禁用农药仍屡禁不止。这些现象反映出农业生产源头监管的复杂性和困难性。

农产品流通过程中，食品安全检测是保障食品质量与安全的核心环节，已成为破解农残监管的关键技术。随着科技的发展，检测手段正在向低成本，实时，非化学反应发展。传统的色谱检测技术需要复杂的环境和化学试剂，单次检测成本高，导致难以在农田中进行高频次的筛查。因此基于光谱的检测方法逐渐兴起，它具有无损、高效、灵敏的特点 [1]，能够对农残进行实时监测。然而光谱数据在实际采集过程中常常受到多种因素的干扰，例如环境波动、测量设备以及农作物本身的差异等，这些噪声影响了检测的准确性。为保证检测结果的准确性，通过设计有效的数学模型解决噪声问题，实现不同农药残留的检测，维护食品安全。

1.2 问题要求

现已知数据集：附件 1（待降噪处理数据），附件 2（4 种测量物的反射光谱以及测试集），附件 3（同类测量物不同喷洒次数以及测试集），和附件 4（不同批次喷洒训练集以及测试集）。

根据以上附件数据集并结合具体情况，建立相关的数学模型，解决以下问题。

问题 1：由于给定数据中在红外和紫外部分出现噪声，影响到数据的有效性和准确性。分析提出合适的噪声处理方法，针对附件 1 中的数据进行降噪处理，给出 10 个样本的具体的平滑结果。

问题 2：每种物质的光谱反射率具有不同的特征，根据附件 2 中测量物的反射光谱数据，构建恰当的特征指标。针对测试数据，区分出不同的测量物种类。

问题 3：不同浓度的农药残留在光谱数据中表现出不同的特征，根据附件 3 提供的训练集，建立相关的分类模型来识别不同浓度的农药残留。针对测试集对数据进行分类，给出喷洒次数分类的结果。

问题 4：不同批次数据分布会受到多因素影响，形成不一致的分布。以附件 4 提供的样本数据为基准，分析不同批次数据的异同，提取出关键性的不变特征。并设计一个通用的三分类算法来识别不同喷洒次数，得到测试集的喷洒次数分类结果。

二、 问题分析

2.1 问题一分析

对于问题一，要求对样本中给定的光谱数据进行噪声处理，针对红外和紫外部分存在的噪声，提出噪声处理方法，得到样本的平滑结果，属于数据预处理问题。首先，我们对数据进行读取，筛选出紫外和红外部分的数据，进行归一化处理。将处理好的光谱数据可视化标记，观察噪声区间，选取需要去除噪声的波长区段。接着，考虑光谱信号通常具有较为平滑的变化和一些局部波动的特点，以及噪声表现为快速变化的高频成分。我们选择 **Savitzky-Golay (SG) 平滑滤波器**作为噪声处理的模型，对数据应用平滑处理。最后，输出平滑结果并与原始数据光谱进行对比分析，同时采用**信噪比（SNR)** 对选取的原始数据的去噪区间进行验证。

2.2 问题二分析

对于问题二，要求对同类测量物构建出特征指标，区分不同测量物，属于分类问题。已知光谱数据通常包含多个波段的反射率信息，利用不同测量物的光谱特征存在差异，同类物体的光谱数据具有相似性这些特点，通过特征提取与选择，从光谱数据中识别出最具代表性的部分，为不同测量物构建出有效的特征指标，用来区分不同的测量物种类，帮助后续的分类或回归分析。我们需要建立数学模型提取出能够有效区分各类物质的特征，形成特征空间，根据要求对附件 2 中的不同测量物进行分类。

2.3 问题三分析

对于问题三，根据训练数据集包含的不同农药喷洒次数的光谱数据，我们要构建一个能够处理高维数据，去除冗余信息并有效提取出特征变量的识别模型，达到对农药喷洒次数进行准确分类的目的。所以问题三是一个典型的多分类问题。首先，已知光谱数据中存在噪声，我们将通过标准化对原始数据进行归一化的预处理，并添加相应的类别标签。接着，我们需要考虑如何有效地从大量波长数据中提取有用的特征，确保模型能够处理光谱数据中的复杂非线性关系，并在测试集上达到较高的分类精度。最后优化特征提取和选择过程，提高模型对农药喷洒次数的预测能力。

2.4 问题四分析

对于问题四，核心是如何应对跨批次数据的分布不一致性，降低多因素的影响，做到准确预测农药的喷洒次数。附件 4 中两个批次的数据集，具有相同的农药浓度，不同的样本和喷洒间隔，导致数据的特征受到多种因素的影响。我们需要构建通用的三分类模型，使其能够有效地识别和利用这些异质数据。模型构建的关键点是采用相适应的方法对数据预处理，跨批次的特征提取和通用分类算法的设计。

三、 模型假设

为简化问题，本文做出以下假设。

- 假设 1：所有噪声是可以通过合适的降噪方法消除的。
- 假设 2：不同种类的测量物的光谱特征具有稳定性，即在实验条件变化不大的情况下，光谱数据的波动较小。
- 假设 3：不同浓度的农药在光谱数据中呈现出明显且可区分的特征，即能够通过一定的特征提取方法识别出不同农药喷洒次数。

四、 符号说明

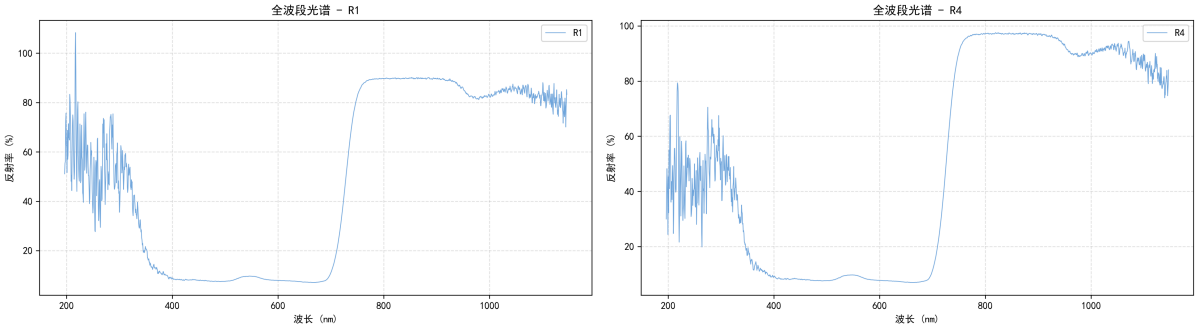
符号	说明
$y_s(i)$	平滑后第 i 点反射率
$window_length$	窗口宽度
$polyorder$	拟合多项式阶数
SNR	信号功率与噪声功率的比值
$R_i(\lambda)$	样本 i 在波长 λ 处的原始光谱反射率函数
$\hat{R}_i(\lambda)$	样本 i 在波长 λ 处平滑处理后的反射率函数
Λ_k	第 k 个光谱波段
$\mathbf{x}^{(i)}$	样本 i 的特征指标
P, Q	载荷矩阵，描述原始变量与潜变量的关系
E, F	残差矩阵，表示未能通过潜变量解释的部分
n_{comp}	最优潜变量成分数
$\hat{y}_{\text{cont}}$	数据集的连续预测值
$\hat{y}_{\text{int}}$	离散化后的预测类别

五、 问题一的模型的建立和求解

在光谱数据采集的过程中，样本自身因素以及周围环境的干扰会严重影响数据的准确性。其中，高频噪声会扰乱吸收峰位置和整体形态等有效信息，掩盖潜在的重要特征。因此，我们需要采取有效的数据处理方法来去除这些噪声，降低干扰信息的影响，保留数据的主要特征，从而提高光谱数据的准确性。

5.1 模型建立

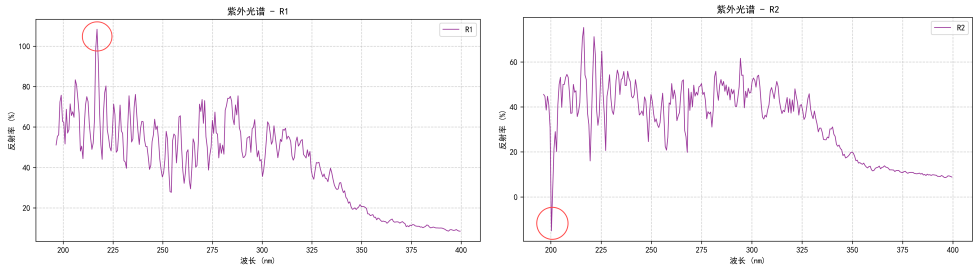
通过对所有 10 个样本的全波段光谱图进行分析，如图 1所示，我们发现紫外光波段整个区域的反射率均呈**锯齿状波动**，有明显的噪声；红外光谱在波长 700-800nm 部分曲线较为平滑，而在波长 800-900nm 部分波动现象不明显，因此我们选取紫外光波段与波长在 900nm 以上的红外光波段数据进行噪声去除。



注：其他样本的全波段原始折线图与本例相似

图 1　部分样本的全波段原始折线图

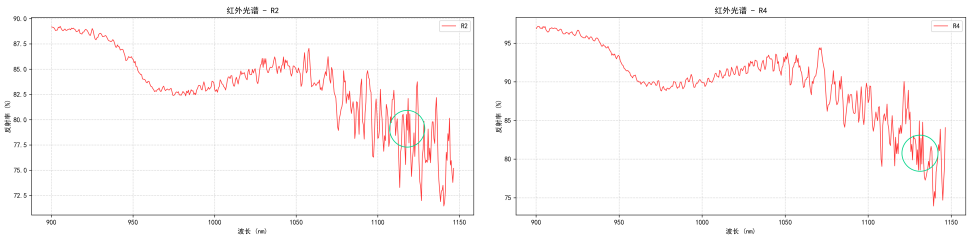
通过分析紫外波段光谱图 (见图 2) 和红外波段光谱图 (见图 3)，我们也可以明显地观察到数据在紫外部分存在超出真实反射率范围的数值、在红外波段存在锯齿状波动的情况。为了有效保留光谱的峰值信息并减少噪声的影响，我们选择使用 **Savitzky-Golay (SG) 平滑滤波器**对光谱数据进行平滑处理。



(a) R1　　**(b) R2**

注：其余样本在此波段的光谱图与所选样例相似

图 2　紫外波段折线图



(a) R2 **(b) R4**

注：其余样本在此波段的光谱图与所选样例相似

图 3　红外波段折线图

Savitzky-Golay(SG) 平滑滤波器是一种基于局部多项式拟合的平滑方法。它通过在滑动窗口内对光谱数据点进行多项式拟合，并用拟合值代替中间点，达到平滑光谱数据的效果。与简单滑动平均方法对比，SG 滤波器能够更好地保留光谱数据的原始形状和趋势，有助于去除高频噪声并保持信号的局部结构 [2]。

SG 滤波公式如下：

$$y_s(i) = \sum_{j=-m}^{m} c_j \cdot y(i+j) \tag{1}$$

其中：$y(i)$ 表示原始信号第 i 点，$y_s(i)$ 表示平滑后第 i 点，c_j 表示多项式拟合的系数，$2m+1$ 表示窗口宽度 w。

SG 滤波的关键参数是窗口宽度 w 和多项式系数 p。窗口宽度 w 决定了滑动窗口包含的数据点数。窗口选择越大，平滑效果越强，但可能会导致光谱细节的丢失。而多项式阶数 p 表示拟合多项式的阶数，决定了拟合曲线的灵活性，选择合适的 p 值能有效保留数据的变化结构。文献 [3][4][5] 指出，p 的取值一般为 2,3,4，而 w 的选取满足 $w > p$ 且为奇数。在实验中，我们不断调整窗口宽度 w 与阶数 p 的取值来优化平滑效果。

为了量化并对比不同参数对光谱的降噪效果，本文引用了**信噪比（SNR）**作为滤波效果评估指标。公式为：

$$SNR = \frac{\mu(y_{\text{smooth}})}{\sigma(y_{\text{raw}} - y_{\text{smooth}})} \tag{2}$$

其中：y_{smooth} 表示 SG 滤波平滑后的信号，y_{raw} 表示原始光谱信号，$y_{\text{raw}} - y_{\text{smooth}}$ 是残差，视作信号的噪声。

在遍历各个参数组合后，部分参数对应的 SNR 显著高于正常范围，出现极端异常值。为避免影响结果，本研究采用**经验阈值筛选**，剔除了显著偏离总体分布的异常值。

为综合红外与紫外两个波段的平滑效果，本文分别计算两波段各组参数的 SNR，并对结果进行 $Z-score$ 标准化:

$$Z_i = \frac{SNR_i - \mu_{\text{SNR}}}{\sigma_{\text{SNR}}} \tag{3}$$

其中：μ_{SNR} 表示各波段的均值，σ_{SNR} 表示各波段的标准差。

最终综合得分为两个波段标准化分数之和：

$$\text{Score}_{\text{final}} = Z_{\text{infrared}} + Z_{\text{ultraviolet}} \tag{4}$$

基于最终得分，绘制参数优化热力图，不同参数最终得分热力图如下（图 4）：

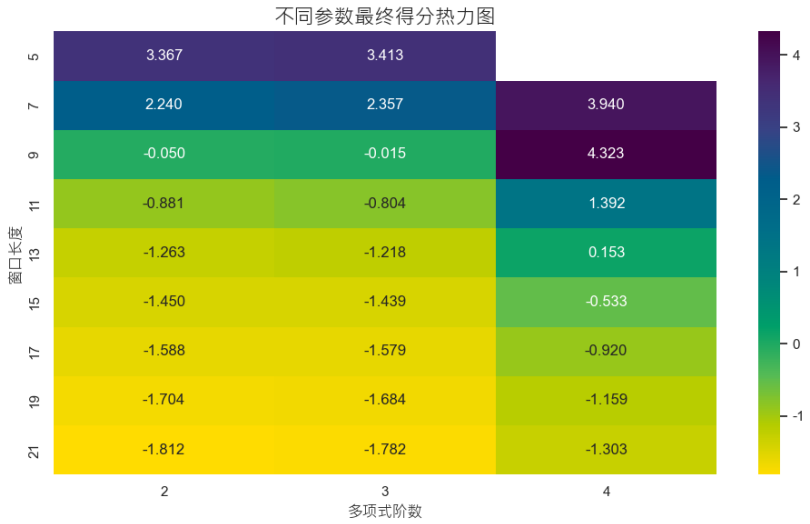


图 4　不同参数最终得分热力图

热力图显示，在剔除异常值后，红外与紫外的最佳参数组合均集中在 **w=9，p=4** 附近。因此，综合考虑信号保真性、噪声抑制及后续模型的泛化能力，本研究将参数设置为 w=9，p=4。

同时，对比多种参数设置，我们也能够发现选择 $w = 9, p = 4$ 时，能够在有效去除高频噪声的同时较好地保留光谱的峰值特征，平衡了平滑效果与形态保留之间的关系[6]。例如图 5所示，当 $w = 5, p = 4$ 时，平滑曲线与原始曲线非常贴近，降噪效果不明显。而如图 6所示，当 $w = 15, p = 4$ 时，平滑曲线与原始曲线的部分趋势出现了较大的差异，部分峰值特征被丢失。

5.2 模型求解

根据上述参数设置，我们对 10 个样本进行降噪处理，部分样本处理结果如图 7所示。通过将处理结果与原始数据进行比较，可以看到经过 SG 平滑滤波后，光谱数据变得更加平滑，随机波动显著减少，尖峰噪声被有效去除。同时，数据的波峰和波谷位置基本保持一致，证明平滑处理没有对光谱的主要结构产生不利影响，能够较好地保留数据中的重要特征。

原始数据中由于噪声引起的短期波动已经被去除，且数据在整体趋势上更为一致，能够更准确地反映样本的实际光谱特征。这对于后续的特征提取和分类分析至关重要。

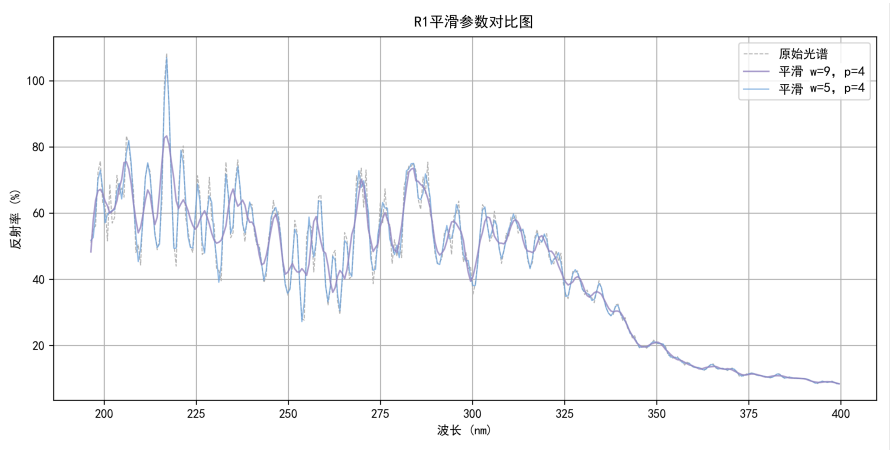


图 5　平滑参数对比 1

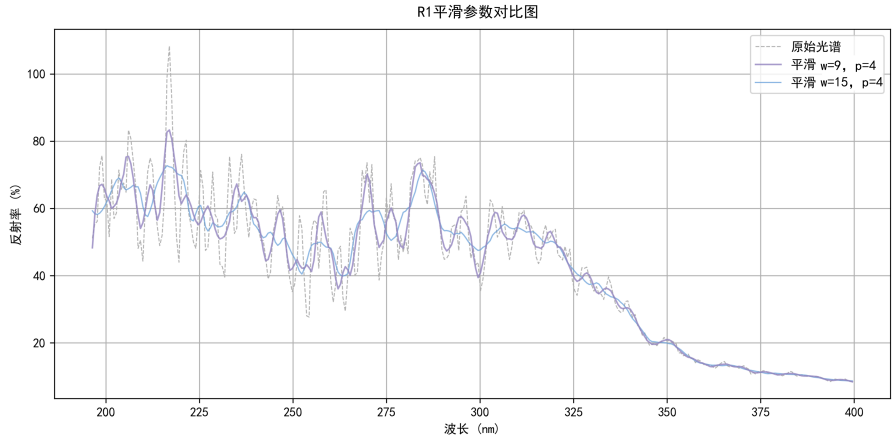


图 6　平滑参数对比 2

5.3 结果分析

为了量化模型去除噪声时对其他有效信息保留的效果，我们引入了**相对残差**作为衡量标准。

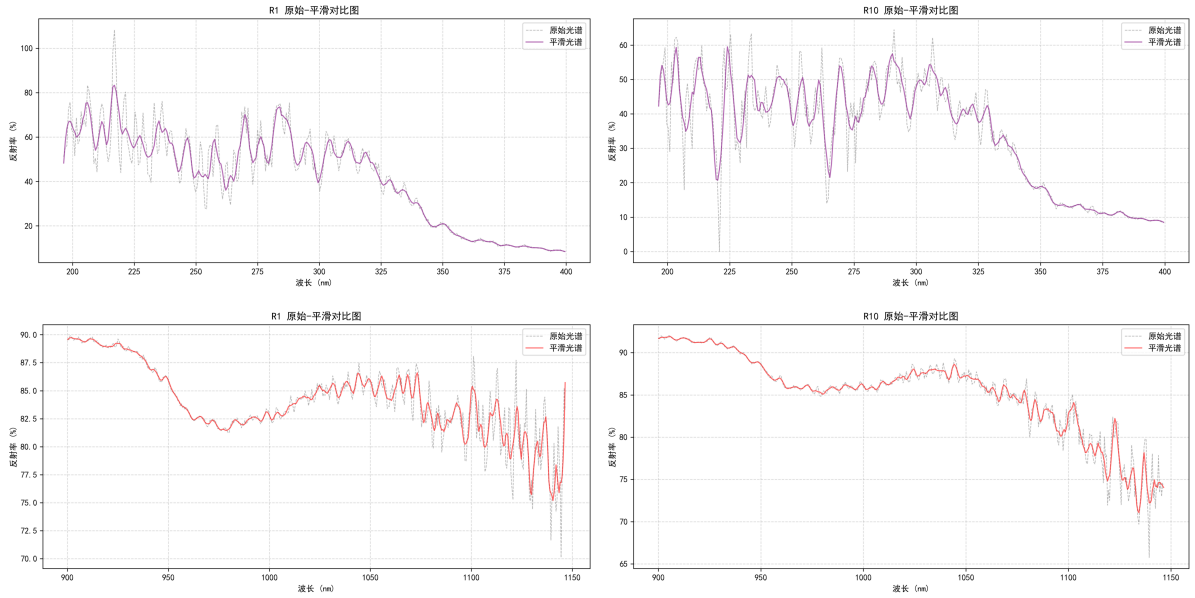
原始标准差的计算公式：

$$\sigma_{\text{original}} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(Y_{\text{original},i} - \overline{Y}_{\text{original}})^2} \tag{5}$$

残差标准差的计算公式：

$$\sigma_{\text{residual}} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(Y_{\text{original},i} - Y_{\text{smooth},i})^2} \tag{6}$$

其中：$Y_{\text{original},i}$ 表示第 i 个波长点的原始反射，$Y_{\text{smooth},i}$ 表示第 i 个波长点的平滑后反射率



注：其他样本的对比图与所选样例相似

图 7　部分样本的原始—平滑对比图

相对残差的计算公式：

$$\text{Relative Residual} = \frac{\sigma_{\text{residual}}}{\sigma_{\text{original}}} \tag{7}$$

根据公式 (5) (6) (7) 计算得到 10 个样本的相对残差数值，如表1 所示。

表 1　相对残差计算结果

样本	相对残差	样本	相对残差
R1	0.349557	R6	0.181752
R2	0.235665	R7	0.189353
R3	0.255194	R8	0.153384
R4	0.184724	R9	0.158547
R5	0.238602	R10	0.185783

根据文献 [7][8] 提出，在近红外光谱分析中，当相对残差值小于 1/3 时，表明平滑处理结果适当，不会导致光谱发生显著的信息损失。通过分析表1 中的数据，可以看出，经过平滑处理后，模型能够有效地去除噪声，同时保留了数据的原始信息。

六、 问题二的模型的建立和求解

6.1 数据预处理

基于问题一的分析，我们发现原始光谱数据中存在高频噪声，这使得测量物的光谱曲线出现波动。同时，由于原始数据的维度过高，又包含大量冗余的波长，容易导致模型过拟合。因此我们首先要对数据进行预处理，尽可能消除噪声对数据的影响。

Step1：Savitzky-Golay 平滑滤波

首先，读取附件二中不同测量物的数据，以波长为新的维度整理各个样本数据。根据问题一的验证结果，我们继续采用 SG 滤波去除光谱数据中的噪声并进行平滑处理。公式为：

$$\hat{R}_i(\lambda) = SG\left(R_i(\lambda),\, w = 11,\, p = 2\right) \tag{8}$$

其中：$R_i(\lambda)$ 表示第 i 个样本在波长 λ 处原始光谱反射率函数，$\hat{R}_i(\lambda)$ 表示平滑处理后的光谱反射率函数。

通过大量实验，我们发现当参数调整为 w=11,p=2 时，模型的泛化性能得到显著提高，并且不会出现过拟合的风险。

Step2：数值裁剪

经过 SG 平滑滤波后，我们发现有部分数据依然超出 [0,100] 的真实光谱反射率范围，属于异常现象。因此这里我们需要剔除异常极值，将光谱反射率限定在合理范围 [0,100] 之间。利用公式：

$$R_{clip}(\lambda) = \begin{cases} 0, & R_{clip}(\lambda) < 0 \\ R(\lambda), & 0 \le R_{clip}(\lambda) \le 100 \\ 100, & R(\lambda) > 100 \end{cases} \tag{9}$$

使得处理后的数据能准确反映出测量物的光谱特征。

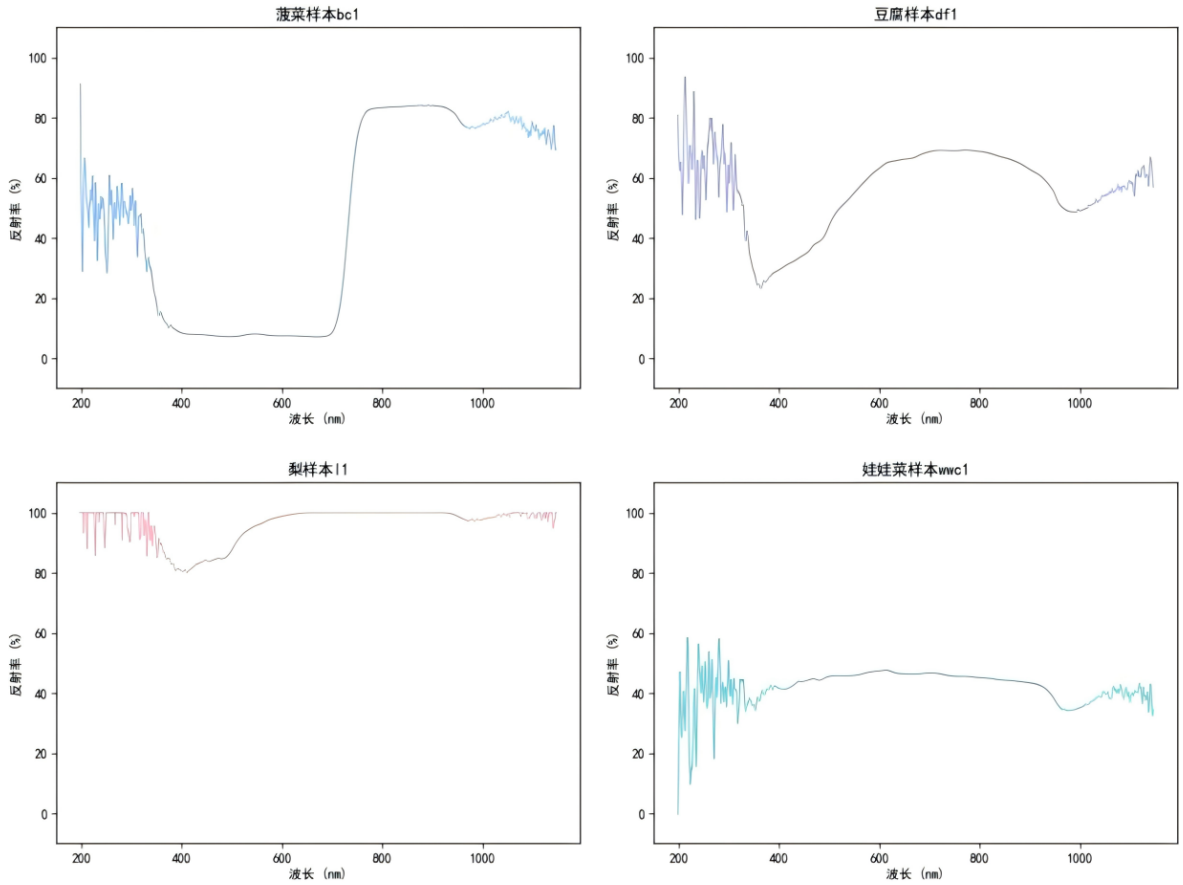
Step3: 数据集划分

将处理后的数据进行整合（每个类别包含 20 个样本，共 80 个样本），为每个样本添加类别标签 (1,2,3,4) 并进行随机打乱，最后将数据集按照 4：1 的比例划分为训练集和验证集。

6.2 特征指标构建

考虑到光谱数据集的高维特性，如果直接使用预处理后的原始数据，会带来维度灾难，导致模型的复杂程度极大增高，严重降低了模型的泛化能力。因此，我们采用了**统计特征提取**的方法，为不同种类的蔬菜构建特征指标，从而有效减少数据维度。

根据图 8所示，我们可以观察到，在波长 200-400nm 的紫外区段，光谱数据的波动明显；在 400-1000nm 的区段，光谱曲线较为平滑；而在 1000nm 以上的红外区域，数据



注：其他各个样本的原始光谱特征图与所选样例相似

图 8　四个样本的原始光谱特征图

中又存在了部分波动。基于对以上四个样本波形图的分析，发现各个区段的曲线特征具有显著差异，因此，本文将波长范围划分为三部分，如表 2所示。

表 2　光谱波长划分表

符号	数学定义	物理意义
Λ_1	$[200nm, 400nm]$	紫外光区，对应电子跃迁
Λ_2	$(400nm, 1000nm]$	可见光-近红外区，含分子振动信息
Λ	$\Lambda_1 \cup \Lambda_2$	全谱区，覆盖主要光谱特征

同样地，通过观察不同品种各个波段的光谱图，我们构建了三个统计特征：均值 (mean)，标准差 (std) 和极差 (range)。

对各个品种所有样本 i 在每个所选波长区间 Λ_k 提取以上特征。

- **均值（mean）**：

$$\mu_{\Lambda_k}^{(i)} = \frac{1}{\Lambda_k} \sum_{\lambda \in \Lambda_k} R_i(\lambda) \tag{10}$$

- **标准差（std）**：

$$\sigma_{\Lambda_k}^{(i)} = \sqrt{\frac{1}{\Lambda_k} \sum_{\lambda \in \Lambda_k} \left(R_i(\lambda) - \mu_{\Lambda_k}^{(i)} \right)^2} \tag{11}$$

- **极差（range）**：

$$\delta_{\Lambda_k}^{(i)} = \max_{\lambda \in \Lambda_k} R_i(\lambda) - \min_{\lambda \in \Lambda_k} R_i(\lambda) \tag{12}$$

其中：i 是样本编号，Λ_k 表示第 k 个光谱波段，$R_i(\lambda)$ 表示样本 i 在波长 λ 处的反射率。

通过上述方法，我们实现了将原始 1573 维的光谱特征降维至 9 维，构建出样本 i 的特征指标，即

$$\mathbf{x}^{(i)} = \big[\underbrace{\mu_{\Lambda_1}, \mu_{\Lambda_2}, \mu_{\Lambda}}_{\text{均值特征}}, \underbrace{\sigma_{\Lambda_1}, \sigma_{\Lambda_2}, \sigma_{\Lambda}}_{\text{标准差特征}}, \underbrace{\delta_{\Lambda_1}, \delta_{\Lambda_2}, \delta_{\Lambda}}_{\text{极差特征}} \big]$$

6.3 模型选择与构建

通过对提取的 9 维统计特征指标进行分析，我们发现样本数据的规模较小，且特征之间存在非线性的组合关系。这些性质使得分类边界呈现非线性的特点，要求模型具有较强的泛化能力，难以采用常规的分类方法来对待测样本进行标签预测，而**支持向量机（SVM）**作为一种机器学习模型，可以通过使用非线性核函数（RBF）将数据映射到高维空间，从而使数据变得线性可分 [9][10]。因此本文考虑使用支持向量机对样本进行分类。使用 SVM 对样本进行分类时，通过对训练集 X_{train} 和它的标签 y_{train} 进行标准化，使得每个特征对模型的贡献程度相当，避免由于不同特征尺度差异对模型训练的影响。

设优化问题为：

$$\begin{aligned} & \min_{\boldsymbol{w}, b, \boldsymbol{\xi}} \frac{1}{2} \|\boldsymbol{w}\|^2 + C \sum_{i=1}^{n} \xi_i \\ \text{subject to} \quad & y_i(\langle \phi(\boldsymbol{x}_i), \boldsymbol{w} \rangle + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \quad \forall i \in \{1, \ldots, n\} \end{aligned}$$

通过核函数 $K(x_i, xj)$ 代替内积计算，使用径向基核函数 (RBF):

$$K(X_i, X_j) = \exp\left(-\gamma \|X_i - X_j\|^2\right) \tag{13}$$

将光谱数据映射到高维空间后，进行**超参数调节**。C 参数能够控制模型对误差的容忍度。当 C 较大时，模型会更倾向于减少训练错误，可能导致过拟合；当 C 较小时，模型的间隔较大，能容忍更多错误。γ 参数用于控制 RBF 核的宽度，较大的 γ 会使得决策边界更复杂，可能导致过拟合。因此本文使用了交叉验证来调节这些超参数，确保模型能够在未见数据上保持良好的性能。绘制 SVM 的大致结构图如图 9。

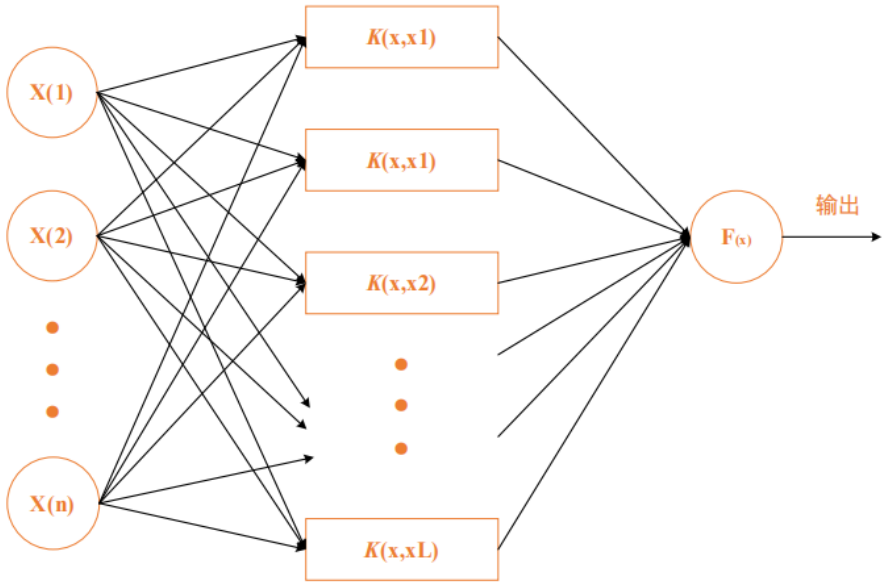


图 9　SVM 结构图

6.4 模型求解与验证

6.4.1 求解结果

将附件 2 中的待测样本数据输入至模型中，通过 SVM 模型得到分类结果，如表 3所示。

表 3　样本分类结果统计

样本编号	预测类别	样本编号	预测类别
待测种类 1	娃娃菜	待测种类 11	豆腐
待测种类 2	菠菜	待测种类 12	豆腐
待测种类 3	梨	待测种类 13	梨
待测种类 4	菠菜	待测种类 14	菠菜
待测种类 5	豆腐	待测种类 15	娃娃菜
待测种类 6	娃娃菜	待测种类 16	豆腐
待测种类 7	豆腐	待测种类 17	娃娃菜
待测种类 8	梨	待测种类 18	菠菜
待测种类 9	菠菜	待测种类 19	梨
待测种类 10	梨	待测种类 20	娃娃菜

6.4.2 模型验证

将测试集数据输入至 SVM 模型中，对分类结果进行评估。如表 4所示，表明各类样本均被准确识别。

表 4　各类别表现

类别	样本量	精确率	召回率	F1 分数
1	5	1.0000	1.0000	1.0000
2	6	1.0000	1.0000	1.0000
3	2	1.0000	1.0000	1.0000
4	3	1.0000	1.0000	1.0000

接着，本文利用**主成分分析（PCA）**找到光谱数据中最具代表性的方向。PCA 作为一种数据降维的技术，将原始数据投影到数据变异最大的两个方向上，得到一个二维空间中的数据表示。SVM 根据降维后的训练数据来找到最优的分类边界，这个边界能够最大化类别之间的间隔，将不同类别的数据点分开。

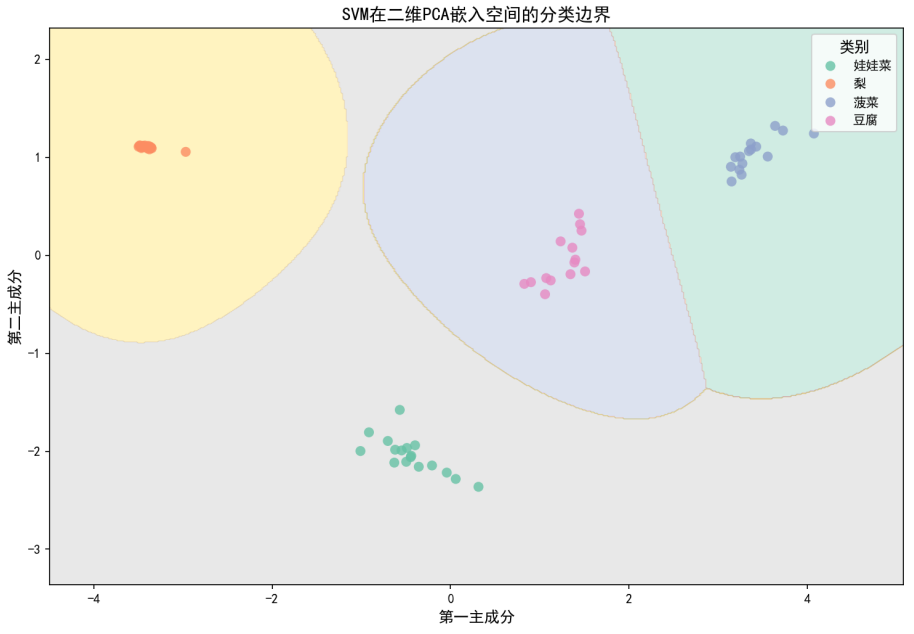


图 10　SVM 在二维 PCA 嵌入空间的分类边界图

如图 10所示，基于 PCA 降维的二维散点图显示：

（1）四类蔬菜在第一、第二主成分空间中各自形成了明显的聚类，且类别之间无严重的重叠交叉区域。验证了所提取的 9 维统计特征具备较好的线性可分性，这为后续支持向量机的分类提供了可靠的特征基础。

（2）分类边界是弯曲的反映了 SVM 在空间中形成了非线性的划分，体现出 SVM 较强的非线性拟合能力。

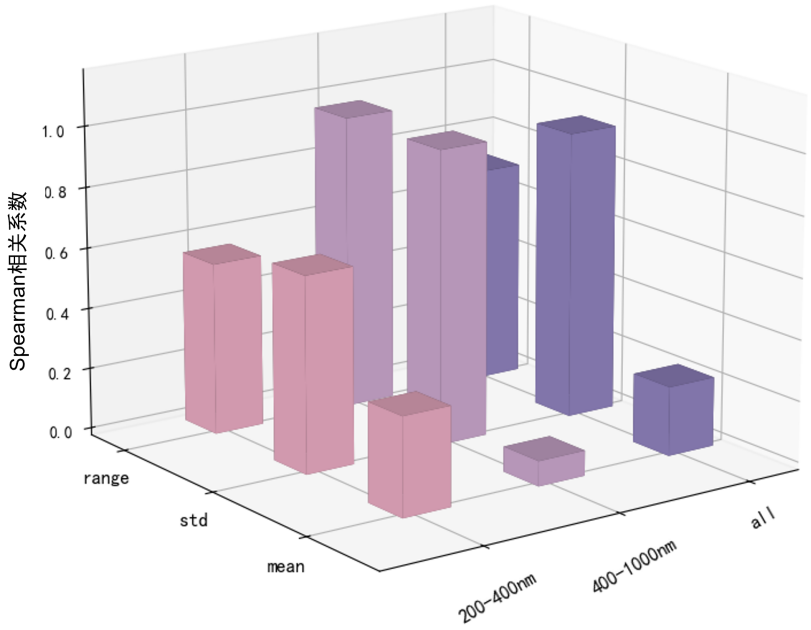


图 11　Spearman 柱状图

此外，在支持向量机这种复杂的非线性回归模型中，**Spearman 秩相关系数**能有效捕捉到非线性关系的强度。本文通过计算 Spearman 秩相关系数来衡量模型的预测类别与真实类别之间的关系。

由图 11所示，图中每个柱子代表一个 Spearman 秩相关系数，分析数据可以看出，模型在波长 400-1000nm 范围内的标准差与极差这两个特征对预测准确性的贡献程度最大，而在该范围内的均值特征对模型准确性的贡献程度不高。

七、 问题三的模型的建立和求解

7.1 数据预处理

异常值截断: 原始光谱反射率数据范围应该为 0-100（%），由于光谱数据的波长范围和幅度差异较大，个别光谱数据受到仪器噪声的影响，导致部分波段反射率超出物理范围，因此我们采用式 (9)双边界截断对数据进行裁剪处理，在消除差异的同时保留光谱的形态特征，降低对特征提取的影响，并且确保数据符合光学反应特性。

Savitzky-Golay 滤波: 为抑制高频噪声，采用问题一提出的 Savitzky-Golay 滤波，如式 (1)，保证在平滑噪声时，做到保留光谱曲线的局部特征。

7.2 模型选择与建立

问题三需要根据光谱特征对农药喷洒次数（0 次、1 次、2 次）进行分类。本文采用以下两阶段对第二问的 SVM 模型进行优化使用：

首先，预处理参数优化，通过 Savitzky-Golay 滤波消除光谱噪声，其窗口长度 (w) 和多项式阶数 (p) 的选择直接影响特征质量。

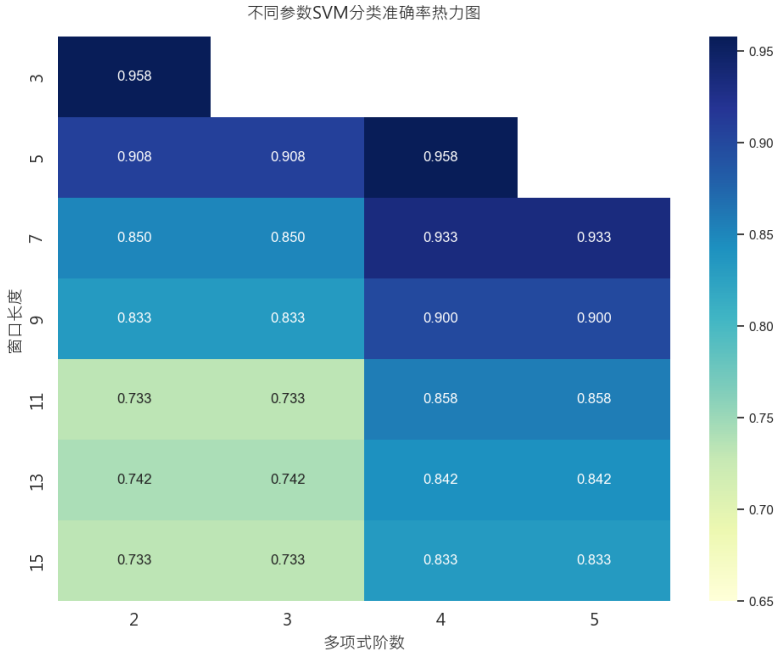


图 12　不同参数 SVM 分类准确率热力图

如图 12所示，Savitzky-Golay 滤波的窗口长度（w）和多项式阶数（p）对模型性能具有显著影响。实验发现，当参数组合为（w=3, p=2）或（w=5, p=4）时，5 折交叉验证准确率均达到峰值。经综合评估计算效率与模型复杂度，最终选择**（3,2）**作为最优配置，因其在保持同等精度的同时，显著降低了运算成本，更符合实际应用需求。

其次，针对光谱数据高维小样本的特性（千维连续反射率特征 + 有限训练样本），核心模型沿用第二题**支持向量机（SVM）**并执行以下优化流程：

(1) 使用方差阈值法移除低方差特征，消除冗余信息干扰。

(2) 通过 5 折交叉验证的网格搜索确定 SVM 最优参数。

(3) 采用分层交叉验证评估泛化性能，最终模型在测试集上达到 **95.8%** 的准确率。

7.3 求解结果与其他模型交叉验证

7.3.1 求解结果

问题三测试集的分类结果如表 5所示。

表 5　测试样本喷洒预测结果

待测试样本 ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
预测喷洒次数	2	0	1	2	1	1	1	0	1	1	0	1	0	1	2

7.3.2 其他模型交叉验证

同时，本研究还通过系统性的模型对比实验，验证了 SVM 在农药喷洒次数分类任务中的优越性能。如表6所示，我们选取了 8 种典型的机器学习模型进行对比分析，所有模型均采用统一的**分层 5 折交叉验证**流程以确保评估的公平性。

实验结果表明，SVM 模型在各项指标上均表现最优。

表 6　机器学习模型性能对比

模型	5 折准确率	平均准确率	5 折 F1 分数	平均 F1 分数
LightGBM	[0.833, 0.917, 0.792, 0.875, 0.833]	0.850	[0.830, 0.915, 0.789, 0.866, 0.833]	0.847
SVM	[0.958, 1.000, 1.000, 0.875, 0.958]	**0.958**	[0.958, 1.000, 1.000, 0.872, 0.958]	**0.958**
RF-RFE	[0.875, 0.875, 0.833, 0.667, 1.000]	0.850	[0.872, 0.877, 0.836, 0.667, 1.000]	0.850
PCA+LR	[0.958, 0.958, 0.958, 0.833, 0.958]	0.933	[0.958, 0.958, 0.958, 0.833, 0.958]	0.933
PLS-DA	[1.000, 0.875, 0.833, 0.875, 0.958]	0.908	[1.000, 0.878, 0.834, 0.878, 0.958]	0.909
XGBoost	[0.750, 0.917, 0.875, 0.667, 0.750]	0.792	[0.747, 0.917, 0.872, 0.658, 0.750]	0.789
Stacking	[0.917, 0.875, 0.792, 0.708, 0.792]	0.817	[0.917, 0.875, 0.788, 0.705, 0.787]	0.814
VAE	[0.792, 0.792, 0.833, 0.583, 0.875]	0.775	[0.790, 0.791, 0.837, 0.544, 0.866]	0.766

注：所有数值保留 3 位小数 | 最优模型 (SVM) 已高亮显示

值得注意的是，PCA+LR 和 PLS-DA 模型分别以 93.3% 和 90.8% 的准确率位列二、三名。这三个最优模型在测试集上的预测结果具有高度一致性，进一步验证了分类结果的可靠性。

八、 问题四的模型的建立和求解

8.1 数据预处理

基于前期问题研究，我们了解到将光谱数据需进行预处理降噪。特别地，由于问题四中数据受批次影响，具有分布异质性（如基线偏移、强度差异等），我们对预处理步骤进行优化与改进。

Step1：归一化处理

将来自不同批次的数据进行整合，由于数据的测量范围和尺度可能存在差异，因此需要进行归一化处理，使数据保持一致，分布在 [0,1] 内。通过公式 (14) 尽可能消除不同批次间测量与环境误差对特征提取的影响。

$$\hat{X}_i = \frac{X_i - \min(X)}{\max(X) - \min(X)} \tag{14}$$

其中：$\hat{X}_i$ 表示第 i 个样本归一化后的值，X_i 表示第 i 个样本的原始值，$\max(X) - \min(X)$ 是极差。

Step2：去噪

在 SG 滤波对数据进行平滑处理基础上应用小波去噪对数据进行再次降噪。小波的多分辨率分析特性对不同频率的噪声有针对性的处理；且光谱中的峰值变化通常是局部信号，小波的时频局部化能力能更好保留峰形信息。

8.2 特征提取

由于光谱数据的高维特性，共线性问题及批次干扰等问题。为构建跨批次通用模型，我们提出**局部细节特征提取策略**，其核心思想是：通过微分运算、局部统计量等**波长不变性特征**，捕捉不同批次共有的光谱形态学模式，而非依赖绝对强度值。基于光谱曲线的一阶、二阶微分特性，提取如表 7 四类鲁棒特征：

表 7　光谱特征提取方法及数学表征

特征类型	数学表达式	物理意义	批次鲁棒性机制
二阶差分	$D^{(2)}_{i,j} = X_{i,j+2} - 2X_{i,j+1} + X_{i,j}$	捕捉光谱曲率变化程度	消除基线偏移的一阶影响
局部方差	$\sigma^2_{\text{local}} = \frac{1}{w}\sum_{k=j}^{j+w}(X_{i,k} - \mu_j)^2$	捕捉局部波段波动强度	窗口统计降低批次偏差
小波能量	$E_{\text{wavelet}} = \sum_{l=1}^{L} \|\mathbf{c}_l\|_1$	表征频域能量分布	多尺度分解隔离批次噪声
导数-积分比	$R = \frac{\|\nabla X\|_1}{\|X\|_1}$	量化信号变化率与总量的关系	比值抵消批次缩放效应

8.3 模型选择与建立

针对光谱数据的非线性特征和批次差异，我们采用 RBF 核函数将数据映射至高维空间，其核宽度参数 γ 设为 0.01。公式为：

$$K(X_i, X_j) = \exp\left(-\gamma \|X_i - X_j\|^2\right), \quad \gamma = 0.01 \tag{15}$$

其中：$K(X_i, X_j)$ 表示样本 X_i 与 X_j 之间的相似度度量，X_i, X_j 表示 d 维特征向量，γ 表示核宽度参数，控制函数衰减速度。

考虑到样本量少、特征维度高的问题，选择**偏最小二乘法判别分析 (PLS-DA)** 作为基础模型，该模型常用于高维数据的分类问题，在光谱分析中十分适用，对数据中存在较多噪声和高度相关特征时具有良好的处理能力。由于光谱数据间存在相关性，PLS 能够通过协方差最大程度地提取信息 [11][12]。有效提取潜变量降低数据维度并建立光谱数据与农药喷洒次数之间的映射关系。数学表达式为：

$$\mathbf{X} = \mathbf{T}\mathbf{P}^T + \mathbf{E}, \mathbf{Y} = \mathbf{U}\mathbf{Q}^T + \mathbf{F} \tag{16}$$

其中：$\mathbf{P}, \mathbf{Q}$ 是载荷矩阵，$\mathbf{E}, \mathbf{F}$ 是残差矩阵。

由于单纯的 PLS-DA 的模型在本题数据特征提取方面表现欠佳，其生成的前三个主成分的 violin 图（如图 13）显示在不同类别间的分布高度重叠，未能有效捕捉判别性特征。且从三维主成分散点图（如图 14）可以直观看出，各类别样本在潜变量空间中混合严重，缺乏明显的聚类趋势，表明该模型对关键特征的提取能力不足，难以实现有效的分类区分。

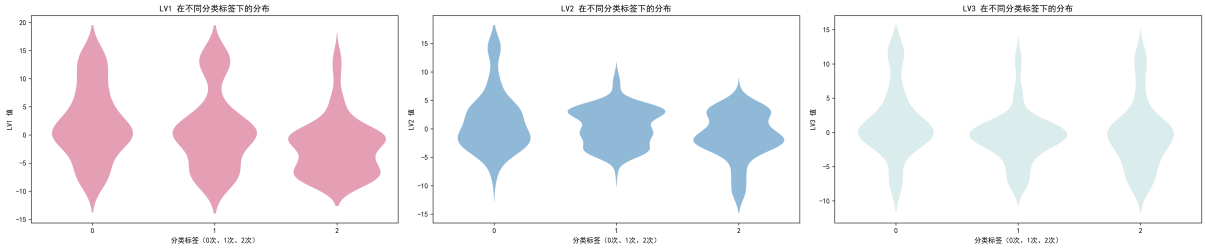


图 13　三个主成分 violin 图

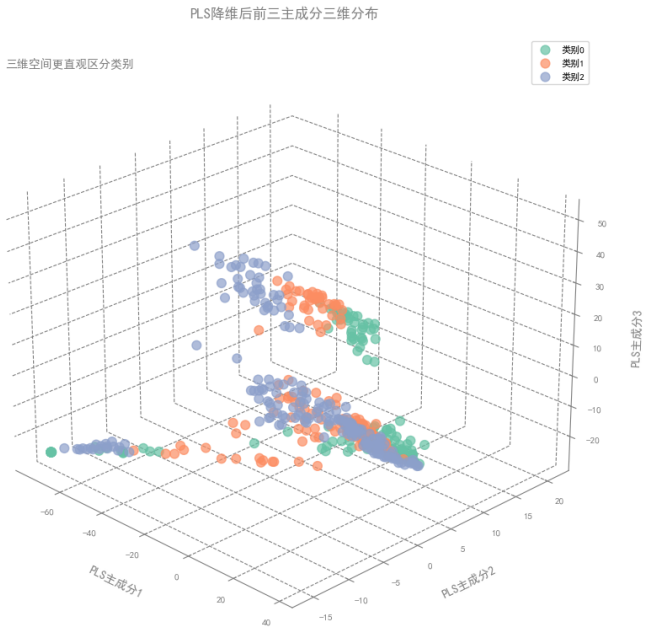


图 14　三维主成分散点图

因此，本题引入了**极限梯度提升模型 XBoost**。其基于梯度提升树算法且通过集成多个决策树模型来捕捉复杂交互（图 15），能够很好地应对此题的非线性关系特征建模，提高预测的准确性。

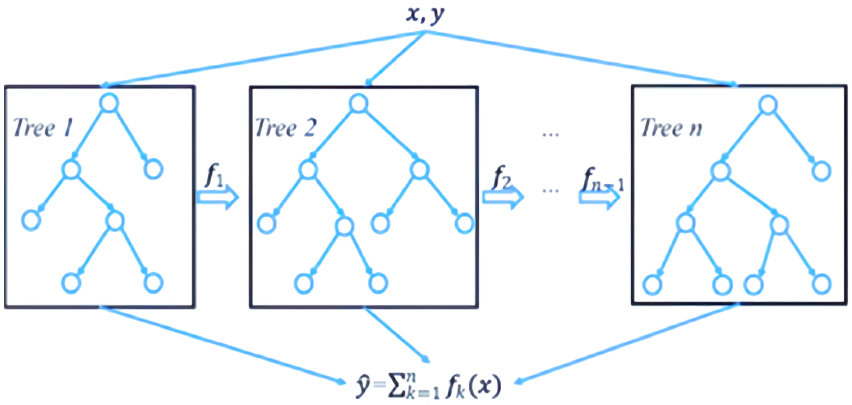


图 15　XGBoost 模型示意图

综合考虑两个模型，我们提出核 **PLS-DA ＋ XGBoost 混合模型**，光谱数据由 RBF 核映射到高维空间，经过 PLS-DA 模型降维后，采用特征融合机制将核空间中的潜变量与原始特征拼接。

建立的完整的模型流程图如图 16 所示。



图 16　菠菜农药喷洒分类模型流程图

8.4 模型求解

根据附件 4 中获得的两批训练数据集，对数据预处理后得到 470 个样本数，初始 1570 个特征数。

Step1：特征选择

通过测试发现，初始确定的四类原始特征（二阶差分，局部方差，小波能量，导数-积分比）都参与训练时，模型预测的准确性只有 **0.3191**，表明原始特征集中依然含有冗余特征。因此我们需要继续选择最相关的特征，目的是减少模型的复杂度。

我们设置不同的组合方式，对四个特征进行实验测试。

实验结果表明，基于不同特征组合的模型预测性能存在差异：当仅采用二阶差分作为特征变量时，模型预测准确率为 0.9489；引入局部方差特征后，准确率提升至 **0.9511**；而进一步增加第三个特征变量时，准确率保持稳定在 0.9511。经过综合评估，我们最终确定采用**二阶差分**、**局部方差**和**小波能量**作为最优特征组合进行模型训练，这一选择在保证预测性能的同时，兼顾了特征的全面性和信息的互补性。模型特征最终选择及维度变化如表 8 所示。

表 8　特征工程方法

特征类型	生成方式	维度变化
二阶差分	np.diff(X, n=2)	1570 → 1568
局部方差	滑动窗口长度为 10	1570 → 1561
小波能量	sym5 小波三层分解	1570 → 4
组合特征	横向拼接	3133 维

Step2：训练参数

使用预处理后的光谱数据及其对应的农药喷洒次数标签训练 PLS-DA + XGBoost 混合模型。在训练过程中，通过五折交叉验证方法优化模型参数，重点针对 PLS-DA 模型的潜变量数量进行筛选，以充分提取数据中的判别性特征，确保模型能够有效地学习数据中的关键特征。同时有效防止模型过拟合，在训练数据上取得最佳性能。经过参数调优实验，最终确定的最优模型参数组合如表 9 所示，该参数配置在训练数据集上表现出最优的分类性能。

表 9　模型训练参数设置表

参数	设置值	计算依据
PLS 主成分数	10	累计解释方差 > 90%
XGBoost 最大深度	3	防止过拟合的折中选择
弱学习器数量	100	早停法验证的最佳迭代次数
学习率	0.1	平衡收敛速度与精度
正则化系数 λ	1	网格搜索验证最优值

Step3：目标函数优化

最后，通过

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \left(\gamma T_k + \frac{1}{2} \lambda \|\mathbf{w}_k\|^2 \right) \tag{17}$$

优化目标函数来求解模型。我们采用迭代优化算法，通过交替优化潜变量数量和其他超参数，如学习率、树深度等，最小化预测误差来优化回归系数，使目标函数 $\mathcal{L}$ 逐步收敛至最小值。最终获得的模型在保证泛化能力的同时，实现了最优的分类性能。

8.5 求解结果

将附件 4 中的测试集输入至模型中，得到预测的分类结果。如图 17 图 18所示。

待分类ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
预测标签	1	2	2	0	1	2	2	0	1	0	1	0	0	1	2	0	2	0	2	2
待分类ID	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
预测标签	2	2	2	0	0	2	1	1	2	0	1	0	0	2	2	0	2	1	0	2
待分类ID	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
预测标签	0	2	1	2	2	1	2	0	2	1	2	2	2	1	2	2	2	1	1	0
待分类ID	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80
预测标签	0	2	2	2	0	2	0	0	0	1	1	2	2	2	0	1	2	1	0	1
待分类ID	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
预测标签	0	0	1	1	2	0	1	2	0	2	2	1	2	2	0	2	0	1	2	0
待分类ID	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
预测标签	0	1	1	0	2	0	2	0	2	1	2	2	1	2	0	2	2	0	1	1

图 17　问题 4 测试集 1 结果

通过分层 K 折交叉验证的方式，评估模型的泛化能力，确保模型在不同数据集上的表现稳定。我们选择 K=5 进行交叉验证，将预处理后的数据集划分为 K=5 个互斥子

待分类ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
预测标签	1	1	2	0	1	2	1	0	2	0	1	1	2	1	2	0	2	0	1	0
待分类ID	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
预测标签	2	2	2	2	0	0	1	1	2	0	1	0	2	2	2	0	2	1	1	2
待分类ID	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
预测标签	0	2	1	2	1	1	2	0	0	1	2	2	2	2	2	1	2	1	1	2
待分类ID	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80
预测标签	0	2	1	0	0	2	0	1	0	1	2	2	0	1	0	1	2	1	2	2
待分类ID	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
预测标签	0	0	1	1	2	0	1	2	0	2	2	2	2	2	0	2	2	1	1	0
待分类ID	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
预测标签	1	1	2	2	0	1	2	0	2	1	0	2	1	2	0	2	2	0	2	1

图 18　问题 4 测试集 2 结果

集，每个子集保持原始数据的类别比例分布。通过这种验证方式，我们能够确保：模型评估结果不受光谱数据划分随机性的影响；每个样本都有机会参与验证；各类别样本在训练和验证过程中得到均衡表示。根据表10，可以看出模型平均准确率达到 95.11%, 证明混合模型对于不同喷洒次数（0 次、1 次、2 次）的判别能力具有很好的泛化性。

表 10　交叉验证结果

折数	第 1 折	第 2 折	第 3 折	第 4 折	第 5 折	平均
准确率 (%)	95.74	94.68	100.00	95.74	89.36	95.11

为了验证所构建模型的分类效果及低维空间中的可分性，本文对提取的高维特征进行了 **UMAP 降维**至二维，并基于降维结果训练 XGBoost 分类器。通过在 UMAP 空间中可视化决策边界，直观展示了模型对不同类别的判别效果。

如图 19所示，背景颜色代表模型在该区域的预测类别，数据点表示实际样本在降维空间中的位置。从可视化结果可以看出，不同类别在二维空间中表现出较明显的区分性，类别 1 和类别 2 在低维空间中形成了较为集中的聚类区域，类别 0 则在部分区域与类别 1 存在一定的重叠。整体来看，XGBoost 能够划分出复杂的非线性决策边界，较好地区分不同类别样本。

此外，部分样本位于决策边界附近，说明该类样本判别难度较大，边界存在一定模糊性。这一现象与高维空间中类别间的复杂关系相符。整体而言，该结果表明模型具备较好的分类性能，同时通过可视化增强了模型的可解释性。

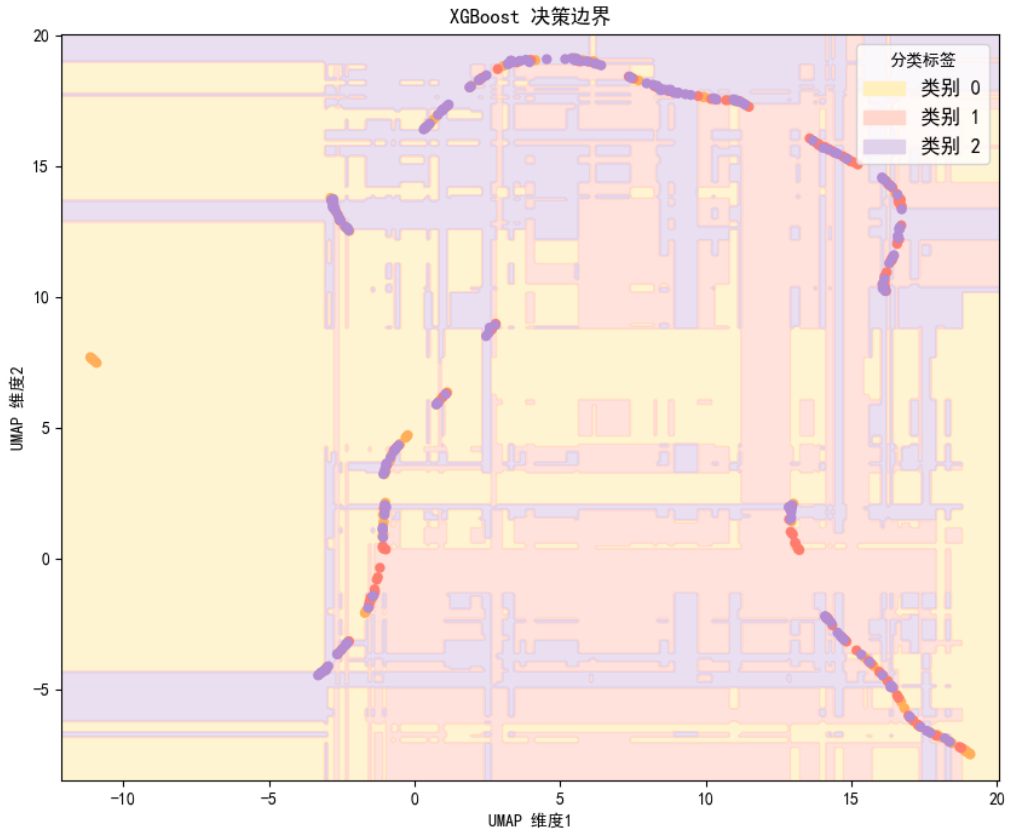


图 19　XGBoost 决策边界

九、 模型的评价

9.1 模型的优点

- **针对性强，充分利用数据特性**

模型在每个问题中都根据光谱数据的特性采用了相应的处理方法，例如在问题一中选用了 Savitzky-Golay 平滑滤波，有效兼顾了去噪和保持光谱峰值的平衡，展示了较强的问题适应性。

- **算法选择合理，具备复杂性与可解释性**

在分类任务中选择了支持向量机 (SVM)、PLS-DA、XGBoost 等机器学习方法，既考虑了数据的非线性与高维特性，也兼顾了模型的可解释性和泛化能力，避免了使用简单线性分类器的不足。

- **特征工程设计有针对性，体现领域理解**

特征提取过程不仅采用了统计特征，如均值、标准差、极差，还结合了微分、局部方差、小波能量等鲁棒特征，从光谱的物理意义出发，有效提高了模型对批次间

差异的适应性。

- **实验验证充分，参数选择有依据**

每个模型都通过交叉验证、网格搜索等方式调优参数，并提供了不同参数组合下的性能对比，如 SVM 核函数参数、SG 滤波窗口和多项式阶数的选择，有实验支持结果，增强了型的说服力。

9.2 模型的缺点

- **数据预处理步骤偏繁琐，自动化不足**

各问题的预处理流程中多次人工调整参数，如 SG 滤波中 w 和 p 的设置、特征选择的组合测试若在实际应用中需要快速处理新数据，缺乏一套自动化、可迁移的流程。

- **特征解释的深度有限**

尽管模型中设计了多种特征，但对于不同特征在物理意义上与农药残留之间的关系、与检测效果的具体对应，讨论仍较为笼统，未能深入揭示“为何这些特征有效”。

- **模型集成方案复杂度较高，实际部署难度大**

在问题四中采用了核 PLS-DA+XGBoost 混合模型，虽然提高了准确率，但该集成方案在实际部署时对计算资源、运行时间和工程实现提出了更高要求，可能限制了模型的推广性。

参考文献

[1] 王丹, 栾雨晴, 谭佐军, 等. 基于高光谱和卷积神经网络的西兰花农药残留检测[J]. 食品工业科技, 2025, 46(6):1-8.

[2] 黄慧波, 李攀, 梁超, 等. 两种优化小波阈值算法对紫外可见光谱的水质 COD 数据去噪研究[J]. 水资源研究, 2022, 11(2):187-193.

[3] SAVITZKY A, GOLAY M J E. Smoothing and differentiation of data by simplified least-squares procedures.[J]. Analytical Chemistry., 1964, 36(8):1627-1639.

[4] W. L, H. Z. Application of sg filter in spectral denoising: effect of polyorder[J]. 2020.

[5] Y. C X W. influence of window length and polyorder in sg filter for soilspectral data.[J]. 2018.

[6] 刘炜, 常庆瑞, 郭曼, 等. 不同尺度的微分窗口下土壤有机质的一阶导数光谱响应特征分析[J]. 红外与毫米波学报, 2011, 30(4):316-321.

[7] YANG Z, LIU X, et al. Evaluation of smoothing and pretreatment methods for near-infrared spectroscopy[J]. Food Chemistry, 2011.

[8] LAVINE B K, WORKMAN J. Chemometrics[J]. Analytical Chemistry, 2006.

[9] 张翠萍, 张军惺, 刘业炜. 应用 MDS-SVM 算法和拉曼荧光光谱技术快速鉴定天麻的掺假行为[J]. 光散射学报, 2025, 37(01):86-93.

[10] 张静, 齐国红, 陈景召. 基于三维荧光光谱和 ISSA-SVM 的食用植物油鉴别[J]. 食品与机械, 2024, 40(10):53-61.

[11] 褚小立, 许育鹏, 陆婉珍. 用于近红外光谱分析的化学计量学方法研究与应用进展[J]. 分析化学, 2008(05):702-709.

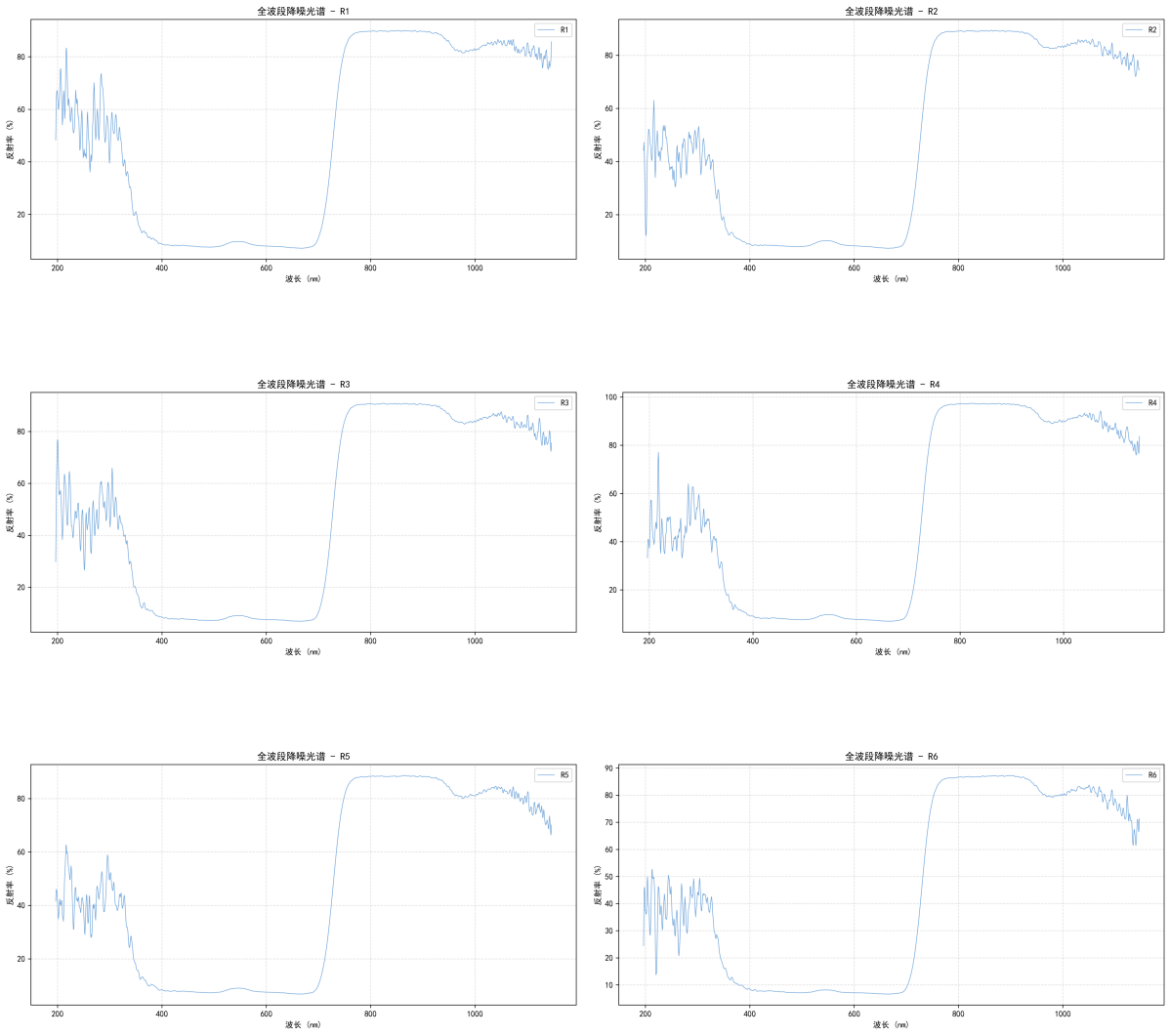
[12] 任博申, 杨林森, 唐必超. 亚高山典型泥炭湿地植被光谱特征分析[J]. 植物科学学报, 2025:1-23.

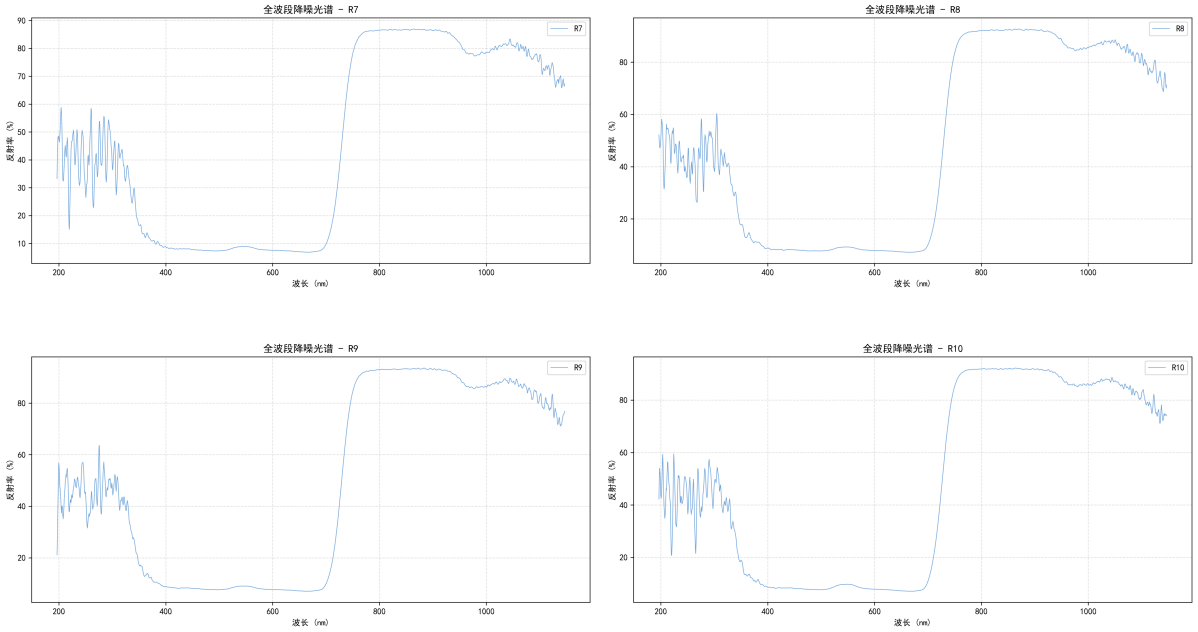
附录 A　文件列表

文件名	功能描述
q1 结果	10 个样本具体平滑结果
q1.py	问题一的核心代码
q2.py	问题二的核心代码
q3.py	问题三的核心代码
q4.py	问题四的核心代码

附录 B　图片

q1 结果 |10 个样本具体平滑结果





附录 C　代码

q1.py| 求解问题一的核心代码

```
import pandas as pd
from scipy.signal import savgol_filter
import matplotlib.pyplot as plt
import os

# 设置字体以支持中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 读取全波段光谱数据
df_uv = pd.read_excel('all_spectrum.xlsx')

# 重命名列：第一列为波长，其余为样本 R1...R10
df_uv.columns = ['Wavelength'] + [f'R{i}' for i in range(1,
    len(df_uv.columns))]

# SG 滤波参数
window_length = 9
polyorder = 4
```

```

# 输出目录
output_dir = '全波段平滑图'
os.makedirs(output_dir, exist_ok=True)

# 对每个样本画平滑后的折线图
for sample in df_uv.columns[1:]:
    y_raw = df_uv[sample].values
    y_smooth = savgol_filter(y_raw, window_length=window_length, polyorder=polyorder)

    plt.figure(figsize=(10, 5))
    plt.plot(df_uv['Wavelength'], y_smooth,
             label=sample,
             color=(97 / 255, 156 / 255, 217 / 255),
             alpha=0.9,
             linewidth=0.75)
    plt.xlabel('波长 (nm)')
    plt.ylabel('反射率 (%)')
    plt.title(f'全波段降噪光谱 - {sample}')
    plt.grid(True, linestyle='--', alpha=0.4)
    plt.legend(loc='upper right')
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, f'{sample}_smoothed.png'), dpi=300)
    plt.close()

print(f"已完成全波段平滑图的生成，保存在目录：{output_dir}")
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import LinearSegmentedColormap
from matplotlib import font_manager
```

```python
from matplotlib.font_manager import FontProperties

# ————— 0. 先准备一份 FontProperties —————
zh_font = FontProperties(family="Microsoft JhengHei", size=14)

# —— 1. 全局 rcParams （让默认字体也走 JhengHei） ——
plt.rcParams['font.family'] = ['sans-serif']
plt.rcParams['font.sans-serif'] = ['Microsoft JhengHei']
plt.rcParams['axes.unicode_minus'] = False  # 负号正常显示

# 设置 seaborn 白色背景
sns.set_theme(style='white')

# 读取数据
df_ir = pd.read_excel('SNR_result_ir().xlsx')
df_uv = pd.read_excel('SNR_result_uv().xlsx')

def remove_outliers(df):
    thresh = df['Mean_SNR'].mean() + 3 * df['Mean_SNR'].std()
    return df[df['Mean_SNR'] < thresh].copy()

df_ir_clean = remove_outliers(df_ir)
df_uv_clean = remove_outliers(df_uv)

df_ir_clean['Norm_SNR'] = (df_ir_clean['Mean_SNR'] -
   df_ir_clean['Mean_SNR'].mean()) / df_ir_clean['Mean_SNR'].
   std()
df_uv_clean['Norm_SNR'] = (df_uv_clean['Mean_SNR'] -
   df_uv_clean['Mean_SNR'].mean()) / df_uv_clean['Mean_SNR'].
   std()

df_merge = pd.merge(
    df_ir_clean[['window_length','polyorder','Norm_SNR']],
    df_uv_clean[['window_length','polyorder','Norm_SNR']],
    on=['window_length','polyorder'],
```

```
    suffixes=('_ir','_uv')
)
df_merge['Final_Score'] = df_merge['Norm_SNR_ir'] + df_merge['Norm_SNR_uv']

pivot_table = df_merge.pivot(index='window_length', columns='polyorder', values='Final_Score')

# 自定义配色
rgb_list = [
    (255/255, 220/255,   0/255),
    (180/255, 204/255,   0/255),
    ( 86/255, 187/255,  85/255),
    (  0/255, 159/255, 105/255),
    (  0/255, 131/255, 125/255),
    (  0/255, 110/255, 137/255),
    (  0/255,  93/255, 138/255),
    ( 56/255,  67/255, 133/255),
    ( 71/255,  39/255, 113/255),
    ( 68/255,   0/255,  70/255),
]
custom_cmap = LinearSegmentedColormap.from_list("custom_rainbow", rgb_list, N=256)

# 绘制热力图
plt.figure(figsize=(10, 6))
sns.heatmap(
    pivot_table,
    annot=True,
    fmt=".3f",
    cmap=custom_cmap,
    linewidths=0,  # 去掉格子线
)

plt.title('不同参数最终得分热力图', fontproperties=zh_font,
```

```
   fontsize=16)
plt.xlabel('多项式阶数', fontproperties=zh_font, fontsize=12)
plt.ylabel('窗口长度', fontproperties=zh_font, fontsize=12)

plt.tight_layout()
plt.show()
```

q2.py| 求解问题二的核心代码

```
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter
import os
import matplotlib.pyplot as plt

# 文件路径（替换文件）
file_path = '娃娃菜数据.xlsx'

# 读取Excel，跳过前4行
df = pd.read_excel(file_path, skiprows=4)

# 获取波长值（第一列）
wavelengths = df.iloc[:, 0].values

# 获取样本数据（去掉第一列）
data = df.iloc[:, 1:].values  # shape = (波长数, 样本数)

# 转置：行=样本，列=波长
data_T = data.T  # shape = (样本数, 波长数)

# Savitzky-Golay 滤波
window_length = 11
polyorder = 2

if window_length >= data_T.shape[1]:
    window_length = data_T.shape[1] - 1 if data_T.shape[1] % 2
```

```
    == 0 else data_T.shape[1]
if window_length % 2 == 0:
    window_length -= 1

smoothed = savgol_filter(data_T, window_length=window_length,
   polyorder=polyorder, axis=1)

# clip 到 [0,100]
smoothed_clipped = np.clip(smoothed, 0, 100)

# 创建 DataFrame，列名用 wavelengths
processed_df = pd.DataFrame(smoothed_clipped, columns=
   wavelengths)

# 加上“样本名”这一列（从原始表头取）
sample_names = df.columns[1:].tolist()
processed_df.insert(0, 'sample', sample_names)

# 增加一列 label=1
processed_df['label'] = 4

# 导出
output_file = 'processed_娃娃菜数据.xlsx'
processed_df.to_excel(output_file, index=False)

print(f'处理完成，已保存：{output_file}')
```

```
import os
import pandas as pd
from sklearn.model_selection import train_test_split

# 1. 配置：文件名与对应标签
file_list = [
    ('processed_菠菜数据.xlsx', '菠菜'),
    ('processed_豆腐数据.xlsx', '豆腐'),
```

```
    ('processed_梨数据.xlsx', '梨'),
    ('processed_娃娃菜数据.xlsx', '娃娃菜'),
]

# 2. 训练/验证集比例（train:val = 4:1 → train_frac = 4/5）
train_frac = 4 / 5


def extract_features(df: pd.DataFrame) -> pd.DataFrame:
    """
    从 df 中提取 200–400nm、400–1000nm、全波段的均值、标准差、极差 9 个特征
    假设 df.columns 包含 'sample'、'label'，其余列都是光谱波长（可识别为数值的字符串或数值类型）
    """
    # 识别光谱列（排除 sample & label）
    spec_cols = [c for c in df.columns if c not in ('sample', 'label')]
    # 将列名映射到数值波长
    col_wls = [(c, float(c)) for c in spec_cols]

    # 按波长范围挑列
    cols_200_400 = [c for c, wl in col_wls if 200 <= wl <= 400]
    cols_400_1000 = [c for c, wl in col_wls if 400 < wl <= 1000]
    cols_all = spec_cols  # 全波段

    # 计算特征
    feats = pd.DataFrame({
        'sample': df['sample'],
        # 均值
        'mean_200_400': df[cols_200_400].mean(axis=1),
        'mean_400_1000': df[cols_400_1000].mean(axis=1),
        'mean_all': df[cols_all].mean(axis=1),
```

```
        # 标准差
        'std_200_400': df[cols_200_400].std(axis=1),
        'std_400_1000': df[cols_400_1000].std(axis=1),
        'std_all': df[cols_all].std(axis=1),
        # 极差
        'range_200_400': df[cols_200_400].max(axis=1) - df[cols_200_400].min(axis=1),
        'range_400_1000': df[cols_400_1000].max(axis=1) - df[cols_400_1000].min(axis=1),
        'range_all': df[cols_all].max(axis=1) - df[cols_all].min(axis=1),
    })
    # 保留标签
    feats['label'] = df['label']
    return feats


# 3. 逐文件读取、提特征并保存
all_feats = []
for fname, label in file_list:
    df = pd.read_excel(fname)
    df['label'] = label  # 添加 label 列

    feats = extract_features(df)
    out_file = f'features_{label}.xlsx'
    feats.to_excel(out_file, index=False)
    print(f'已保存特征文件: {out_file}')

    all_feats.append(feats)

# 4. 合并、打乱、拆分
all_df = pd.concat(all_feats, ignore_index=True)
all_df = all_df.sample(frac=1, random_state=42).reset_index(drop=True)

```

```
train_df, val_df = train_test_split(
    all_df,
    train_size=train_frac,
    random_state=42,
    shuffle=False  # 因为已经 sample 过一次
)

# 5. 导出训练集/验证集
train_df.to_excel('train_set.xlsx', index=False)
val_df.to_excel('val_set.xlsx', index=False)
print('已生成 train_set.xlsx 和 val_set.xlsx')
```

```
# svm_predict_on_features.py

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score,
   classification_report, confusion_matrix

# 1. 读取训练集和验证集
train_df = pd.read_excel('train_set.xlsx')
val_df   = pd.read_excel('val_set.xlsx')

# 2. 确定特征列（训练/验证集里包含 'sample' 和 'label'）
feature_cols = [c for c in train_df.columns if c not in ('
   sample', 'label')]

# 3. 准备训练/验证数据
X_train = train_df[feature_cols].values
y_train = train_df['label'].values
X_val   = val_df[feature_cols].values
y_val   = val_df['label'].values

# 4. 标准化
```

```
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_val_s   = scaler.transform(X_val)

# 5. 训练 SVM
svm_clf = SVC(kernel='rbf', random_state=42)
svm_clf.fit(X_train_s, y_train)

# 6. 验证集评估（可选）
y_val_pred = svm_clf.predict(X_val_s)
print("=== 验证集评估 ===")
print("Accuracy:", accuracy_score(y_val, y_val_pred))
print(classification_report(y_val, y_val_pred, digits=4))
print("Confusion Matrix:\n", confusion_matrix(y_val,
   y_val_pred))

# 7. 读取仅包含 9 个特征的测试集
test_feat = pd.read_excel('testset_features.xlsx')

# 8. 准备测试数据
#    测试表格没有 sample 列，所有列均为特征
X_test = test_feat.values

# 9. 标准化 & 预测
X_test_s    = scaler.transform(X_test)
y_test_pred = svm_clf.predict(X_test_s)

# 10. 保存结果
out = pd.DataFrame({
    'sample_id': range(1, len(y_test_pred) + 1),
    'predicted_label': y_test_pred
})
out.to_excel('test_predictions_svm.xlsx', index=False)
print("已保存测试集预测结果到 test_predictions_svm.xlsx")
```

q3.py| 求解问题三的核心代码

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV,
   cross_val_predict, StratifiedKFold
from sklearn.metrics import accuracy_score, confusion_matrix,
   ConfusionMatrixDisplay, f1_score
from sklearn.feature_selection import VarianceThreshold
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import os

def preprocess_features(X):
    from scipy.signal import savgol_filter
    from scipy.ndimage import gaussian_filter1d
    # 先clip到0-100
    X = np.clip(X, 0, 100)
    # savgol滤波
    X = savgol_filter(X, window_length=3, polyorder=2, axis=1,
    mode='nearest')

    return X

def train_svm(X_train, y_train, model_path='svm_model.pkl', cv
   =5):
    le = LabelEncoder()
    y_encoded = le.fit_transform(y_train)
    selector = VarianceThreshold(threshold=0)
    X_train_selected = selector.fit_transform(X_train)
    param_grid = {
        'C': [0.1, 1, 10],
        'gamma': ['scale', 0.01, 0.1],
```

```
        'kernel': ['rbf']
    }
    svm = SVC(probability=True)
    grid_search = GridSearchCV(svm, param_grid, cv=cv, n_jobs=-1)
    grid_search.fit(X_train_selected, y_encoded)
    best_model = grid_search.best_estimator_
    cv_split = StratifiedKFold(n_splits=cv, shuffle=True, random_state=42)
    acc_list = []
    f1_list = []
    for train_idx, test_idx in cv_split.split(X_train_selected, y_encoded):
        X_tr, X_val = X_train_selected[train_idx], X_train_selected[test_idx]
        y_tr, y_val = y_encoded[train_idx], y_encoded[test_idx]
        best_model.fit(X_tr, y_tr)
        y_pred = best_model.predict(X_val)
        acc = accuracy_score(y_val, y_pred)
        f1 = f1_score(y_val, y_pred, average='weighted')
        acc_list.append(acc)
        f1_list.append(f1)
    mean_acc = float(np.mean(acc_list)) if len(acc_list) > 0 else 0.0
    mean_f1 = float(np.mean(f1_list)) if len(f1_list) > 0 else 0.0
    if model_path is not None:
        joblib.dump({'model': best_model, 'le': le, 'selector': selector, 'cv_acc_list': acc_list, 'cv_acc_mean': mean_acc, 'cv_f1_list': f1_list, 'cv_f1_mean': mean_f1}, model_path)
    return {'model': best_model, 'le': le, 'selector': selector, 'cv_acc_list': acc_list, 'cv_acc_mean': mean_acc, 'cv_f1_list': f1_list, 'cv_f1_mean': mean_f1, 'y_encoded': y_encoded, 'X_train_selected': X_train_selected}
```

```
54
55  def predict_svm(X_test, model_path='svm_model.pkl'):
56      data = joblib.load(model_path)
57      model = data['model']
58      le = data['le']
59      selector = data['selector']
60      X_test_selected = selector.transform(X_test)
61      y_test_pred = model.predict(X_test_selected)
62      y_test_labels = le.inverse_transform(y_test_pred)
63      return y_test_labels
64
65  def plot_feature_importance(model, selector, save_path='SVM_特
    征重要性.png'):
66      if hasattr(model, 'coef_'):
67          feature_importance = np.abs(model.coef_).sum(axis=0)
68          orig_idx = np.where(selector.get_support())[0]
69          full_importance = np.zeros(selector.transform(np.zeros
    ((1, selector.n_features_in_))).shape[1])
70          full_importance[:len(feature_importance)] =
    feature_importance
71          plt.figure(figsize=(10, 6))
72          plt.bar(orig_idx, full_importance)
73          plt.title('Feature Importance (SVM Coefficients)')
74          plt.xlabel('Feature Index')
75          plt.ylabel('Importance')
76          plt.tight_layout()
77          plt.savefig(save_path, dpi=300)
78          plt.close()
79
80  def plot_confusion(y_true, y_pred, labels, save_path=None):
81      conf_matrix = confusion_matrix(y_true, y_pred)
82      disp = ConfusionMatrixDisplay(conf_matrix, display_labels=
    labels)
83      disp.plot()
84      plt.title('Confusion Matrix on Training Set')
```

```
    if save_path:
        plt.savefig(save_path, dpi=300)
        plt.close()
    else:
        plt.show()

def main_train():
    train_df = pd.read_excel('总表.xlsx')
    X_train = train_df.iloc[:, 1:].values
    X_train = preprocess_features(X_train)
    y_train = train_df.iloc[:, 0].values
    result = train_svm(X_train, y_train)
    best_model = result['model']
    le = result['le']
    selector = result['selector']
    acc_list = result['cv_acc_list']
    mean_acc = result['cv_acc_mean']
    f1_list = result['cv_f1_list']
    mean_f1 = result['cv_f1_mean']
    y_encoded = result['y_encoded']
    X_train_selected = result['X_train_selected']
    print(f"交叉验证准确率列表: {acc_list}")
    print(f"交叉验证平均准确率: {mean_acc:.4f}")
    print(f"交叉验证F1列表: {f1_list}")
    print(f"交叉验证平均F1: {mean_f1:.4f}")
    train_pred = best_model.predict(X_train_selected)
    plot_confusion(y_encoded, train_pred, le.classes_)
    train_acc = accuracy_score(y_encoded, train_pred)
    print(f"训练集准确率: {train_acc:.4f}")
```

q4.py| 求解问题四的核心代码

```
# -*- coding: utf-8 -*-
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter
```

```
import pywt
from sklearn.cross_decomposition import PLSRegression
from sklearn.metrics.pairwise import rbf_kernel
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
import os
# ======================== 预处理 ========================
def preprocess(X):
    # 归一化
    X_norm = (X - X.min(axis=1, keepdims=True)) / (X.max(axis=1, keepdims=True) - X.min(axis=1, keepdims=True) + 1e-8)
    # S-G滤波
    X_sg = savgol_filter(X_norm, window_length=11, polyorder=2, axis=1)
    # 小波去噪
    def wavelet_denoise(signal):
        coeffs = pywt.wavedec(signal, 'sym5', level=pywt.dwt_max_level(len(signal), 'sym5'))
        sigma = np.median(np.abs(coeffs[-1])) / 0.6745
        uthresh = sigma * np.sqrt(2*np.log(len(signal)))
        coeffs = [pywt.threshold(c, value=uthresh, mode='soft') for c in coeffs]
        return pywt.waverec(coeffs, 'sym5')[:len(signal)]
    X_denoised = np.apply_along_axis(wavelet_denoise, 1, X_sg)
    print(f"[LOG] 预处理完成，形状: {X_denoised.shape}")
    return X_denoised

# ============ 非线性特征工程 ============
def extract_nonlinear_features(X, wavelengths):
    # 一阶/二阶差分
    diff1 = np.diff(X, axis=1)
    diff2 = np.diff(diff1, axis=1)
    # 局部方差（窗口10）
    window_size = 10
```

```
    local_var = np.array([
        np.convolve(x, np.ones(window_size)/window_size, mode='valid') for x in X
    ])
    # 小波能量（L1范数）
    def get_wavelet_energy(signal):
        coeffs = pywt.wavedec(signal, 'sym5', level=3)
        return np.array([np.linalg.norm(c, ord=1) for c in coeffs])
    wavelet_energy = np.apply_along_axis(get_wavelet_energy, 1, X)
    # 导数-积分比（600-800nm区间）
    idx_600 = np.argmin(np.abs(wavelengths - 600))
    idx_800 = np.argmin(np.abs(wavelengths - 800))
    integral = X[:, idx_600:idx_800+1].sum(axis=1)
    deriv_mean = diff1[:, idx_600:idx_800].mean(axis=1)
    deriv_integral_ratio = integral / (deriv_mean + 1e-8)
    # 合并所有特征
    features = np.hstack([
        diff2,  # 二阶差分
        local_var,  # 局部方差
        wavelet_energy,  # 小波能量
        deriv_integral_ratio.reshape(-1,1)  # 导数-积分比
    ])
    print(f"[LOG] 特征工程完成，输出特征维数: {features.shape[1]}")
    return features

# ========== 核PLS-DA + XGBoost训练 ==========
def train_kplsda_xgb(X, y, gamma=0.01, n_components=10, xgb_params=None):
    # RBF核映射
    K_train = rbf_kernel(X, gamma=gamma)
    # PLS-DA
    pls = PLSRegression(n_components=n_components)
```

```
    T_train = pls.fit_transform(K_train, y)[0]
    # 合并特征
    X_combined = np.hstack([T_train, X])
    # XGBoost
    if xgb_params is None:
        xgb_params = dict(max_depth=3, n_estimators=100, learning_rate=0.1, subsample=0.8, reg_lambda=1)
    xgb = XGBClassifier(**xgb_params)
    xgb.fit(X_combined, y)
    print(f"[LOG] 模型训练完成，PLS主成分: {n_components}, XGBoost参数: {xgb_params if xgb_params else '默认'}")
    return pls, xgb, X, gamma  # X为核参考集
# ========================= 主流程 =========================
if __name__ == "__main__":
    # 读取并合并训练集
    X1, y1, wavelengths = read_data("不同批次异同分析与通用模型训练集/第一批次数据训练集/第一批次总表.xlsx")
    X2, y2, _ = read_data("不同批次异同分析与通用模型训练集/第二批次数据训练集/第二批次总表.xlsx")
    X_train = np.vstack([X1, X2])
    y_train = np.concatenate([y1, y2])
    print("[LOG] 训练集合并完成")
    print(f"[LOG] 训练集X shape: {X_train.shape}, y shape: {y_train.shape}")
    print(f"[LOG] 训练集前5行X:", X_train[:5])
    print(f"[LOG] 训练集前5个y:", y_train[:5])
    print(f"[LOG] 训练集标签类别分布: {dict(pd.Series(y_train).value_counts().sort_index())}")
    # 预处理
    X_train_pre = preprocess(X_train)
    # 特征工程各部分
    diff1 = np.diff(X_train_pre, axis=1)
    diff2 = np.diff(diff1, axis=1)
    print(f"[LOG] diff2 shape: {diff2.shape}")
    window_size = 10
```

```
    local_var = np.array([
        np.convolve(x, np.ones(window_size)/window_size, mode='valid') for x in X_train_pre
    ])
    print(f"[LOG] local_var shape: {local_var.shape}")
    def get_wavelet_energy(signal):
        coeffs = pywt.wavedec(signal, 'sym5', level=3)
        return np.array([np.linalg.norm(c, ord=1) for c in coeffs])
    wavelet_energy = np.apply_along_axis(get_wavelet_energy, 1, X_train_pre)
    print(f"[LOG] wavelet_energy shape: {wavelet_energy.shape}")
    # 训练模型和测试集预测仅用diff2+local_var+wavelet_energy
    train_features = np.hstack([diff2, local_var, wavelet_energy])
    print(f"[LOG] 最终特征组合 shape: {train_features.shape}")
    print(f"[LOG] 训练集前5行特征均值: {np.mean(train_features[:5], axis=0)[:5]}")
    print(f"[LOG] 训练集前5行特征方差: {np.var(train_features[:5], axis=0)[:5]}")
    # 5折交叉验证准确率
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    accs = []
    for i, (train_idx, val_idx) in enumerate(skf.split(train_features, y_train)):
        X_tr, X_val = train_features[train_idx], train_features[val_idx]
        y_tr, y_val = y_train[train_idx], y_train[val_idx]
        model_cv = train_kplsda_xgb(X_tr, y_tr, gamma=0.01, n_components=10)
        pls, xgb, X_ref, gamma = model_cv
        K_val = rbf_kernel(X_val, X_ref, gamma=gamma)
        T_val = pls.transform(K_val)
```

```
119         X_val_combined = np.hstack([T_val, X_val])
120         y_pred = xgb.predict(X_val_combined)
121         acc = accuracy_score(y_val, y_pred)
122         accs.append(acc)
123         print(f"[LOG] 第{i+1}折交叉验证准确率: {acc:.4f}")
124     print(f"[LOG] 最终特征组合(diff2+local_var+wavelet_energy) 5折交叉验证平均准确率: {np.mean(accs):.4f}")
125     model = train_kplsda_xgb(train_features, y_train, gamma=0.01, n_components=10)
126     # 测试集预测与保存
127     def extract_selected_features(X, wavelengths):
128         diff1 = np.diff(X, axis=1)
129         diff2 = np.diff(diff1, axis=1)
130         window_size = 10
131         local_var = np.array([
132             np.convolve(x, np.ones(window_size)/window_size, mode='valid') for x in X
133         ])
134         def get_wavelet_energy(signal):
135             coeffs = pywt.wavedec(signal, 'sym5', level=3)
136             return np.array([np.linalg.norm(c, ord=1) for c in coeffs])
137         wavelet_energy = np.apply_along_axis(get_wavelet_energy, 1, X)
138         feats = np.hstack([diff2, local_var, wavelet_energy])
139         print(f"[LOG] 测试集特征 shape: {feats.shape}")
140         return feats
141     def predict_and_save_selected(model, test_file, output_file, wavelengths):
142         pls, xgb, X_train_ref, gamma = model
143         df_test = pd.read_excel(test_file)
144         X_test = df_test.iloc[:, 1:].values
145         print(f"[LOG] 测试集 {test_file} 样本数: {X_test.shape[0]}, 原始特征数: {X_test.shape[1]}")
146         X_test_pre = preprocess(X_test)
```

```
147         test_features = extract_selected_features(X_test_pre,
    wavelengths)
148         K_test = rbf_kernel(test_features, X_train_ref, gamma=
    gamma)
149         T_test = pls.transform(K_test)
150         X_test_combined = np.hstack([T_test, test_features])
151         print(f"[LOG] 测试集最终输入特征 shape: {
    X_test_combined.shape}")
152         pred = xgb.predict(X_test_combined)
153         df_result = pd.DataFrame({
154             '样本ID': df_test.iloc[:, 0],
155             '预测标签': pred
156         })
157         df_result.to_excel(output_file, index=False)
158         print(f"[LOG] 测试集 {test_file} 预测完成，结果已保存
    到 {output_file}")
159     test_files = [
160         ("通用模型测试集/三分类测试集1.xlsx", "问题4测试集1结
    果.xlsx"),
161         ("通用模型测试集/三分类测试集2.xlsx", "问题4测试集2结
    果.xlsx")
162     ]
163     for test_path, save_path in test_files:
164         predict_and_save_selected(model, test_path, save_path,
     wavelengths)
165     print("[LOG] 所有测试集预测完成")
```